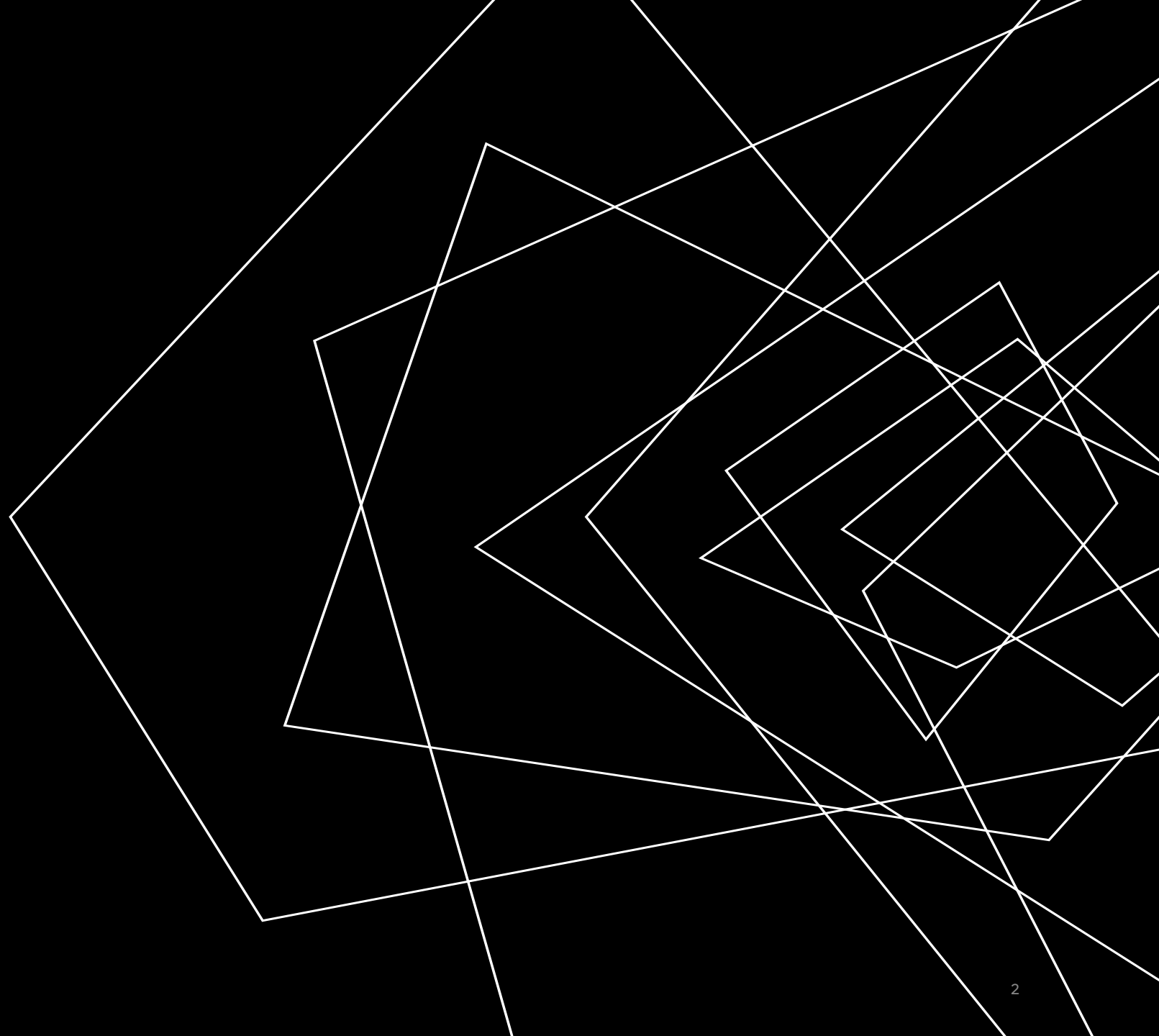
AI IN SOFTWARE DEV:
WORKING WITH AI IN 2026

AGENDA

- History and background
- Agents, tools and models
- My workflow, a live demo and best practices
- Impact and implications



OVERVIEW OF MY JOURNEY

- 2023: early ChatGPT adopter
- 2023-2024: real workflow was still mostly chat, copy/paste, and autocomplete
- Kept following the AI wave and trying new capabilities as they appeared
- Early 2025: experimented with coding agents; results were poor
- Reverted back to chat-based workflows for a while
- Late 2025: the tools crossed a threshold for real development work

WHY I CHANGED MY MIND

- Late 2025, over Christmas break
- Read a Google engineer’s post on using AI for development work
- Tried AI agents seriously on 5+ pet projects and several work projects
- Realized the tools had improved more than I had assumed

WHY THIS TALK

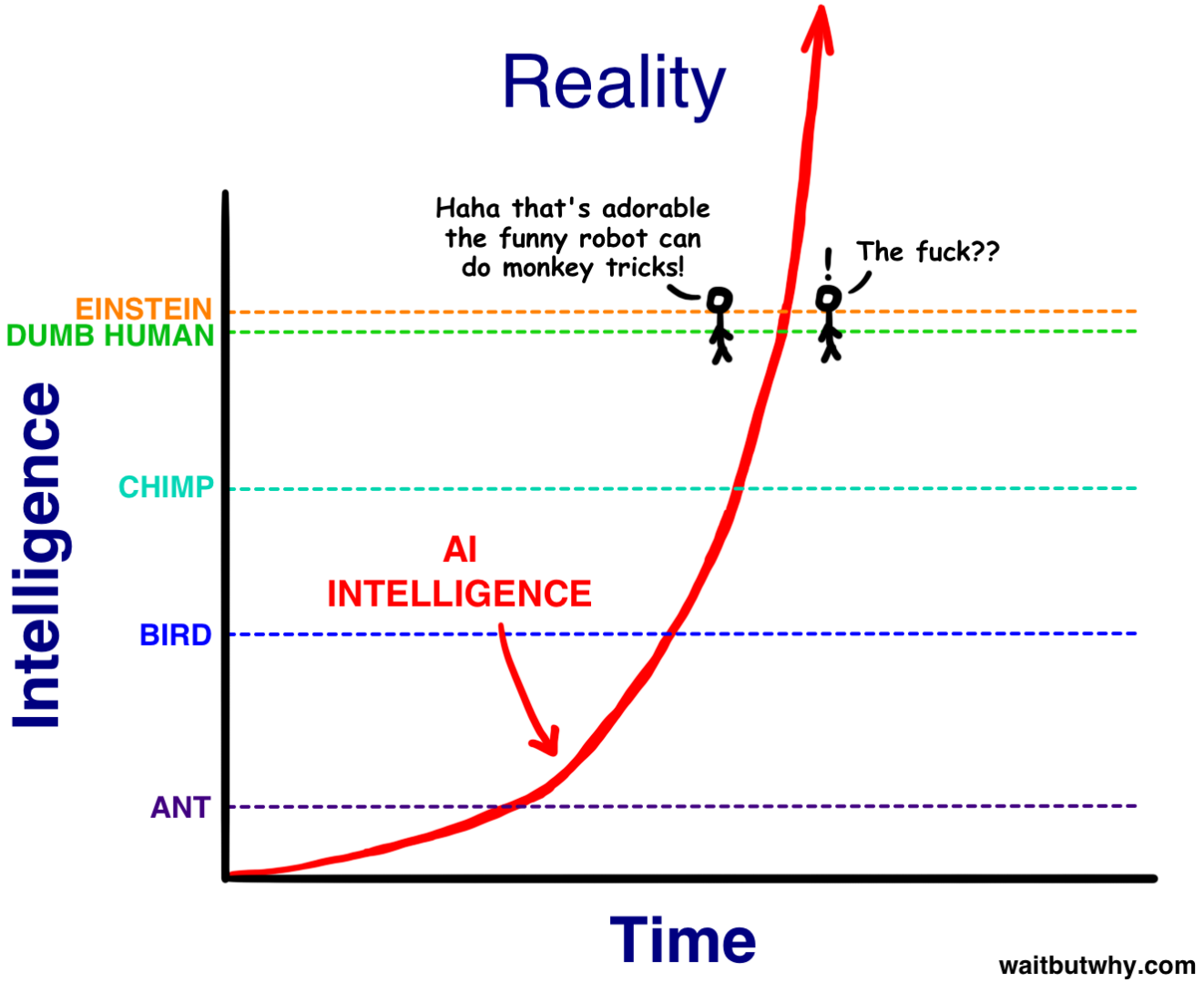
- To help engineers building software/workflows
- A practical report from hands-on use, not a prediction about the future
- The goal is to show:
  - what changed in day-to-day development
  - what workflows actually work
  - what gets better, what gets harder, and what risks increase
  - what this shift means for everyone

THE INFLECTION POINT – NOV 2025

- Hallucination dropped significantly
- Search got much better
- Models needed less micromanaged prompting
- With repo access and enough context, agents became genuinely usable
- November 24, 2025 was described as an event horizon for AI coding
- The shift was not just better answers, but more agentic interfaces
- The tools no longer felt like just chat and autocomplete

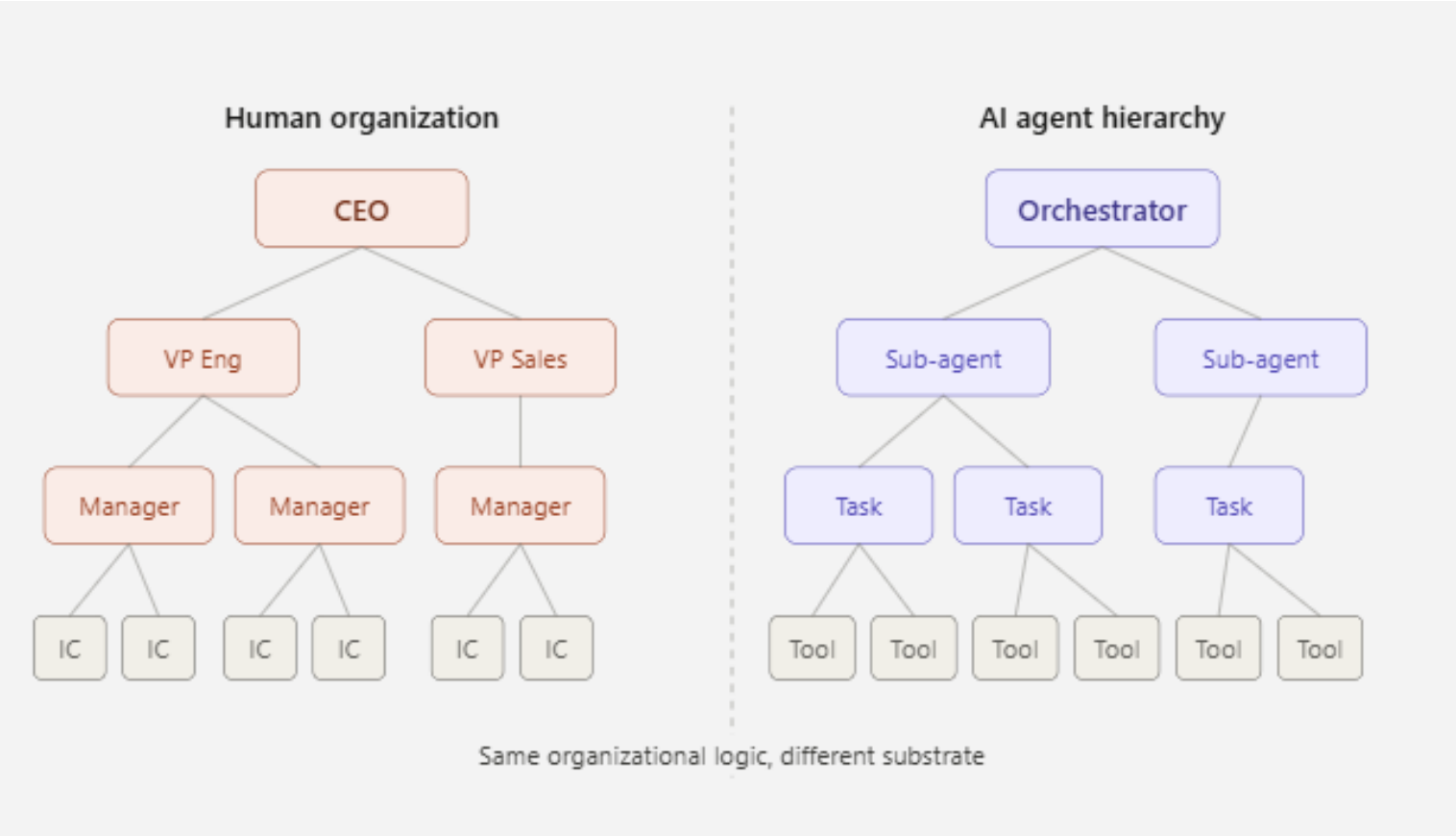
THE INFLECTION POINT – NOV 2025

- 2022: AI could not do basic arithmetic reliably
- 2023: it could pass the bar exam
- 2024: it could write working software
- Late 2025: top engineers said they had handed over most coding work to AI
- February 5, 2026: new models made earlier systems feel like a different era


Reality
Haha that's adorable the funny robot can do monkey tricks!
The fuck??
EINSTEIN
DUMB HUMAN
CHIMP
AI INTELLIGENCE
BIRD
ANT
Intelligence
Time
waitbutwhy.com

THE INFLECTION POINT – NOV 2025

- By early 2026, GPT-5.3 Codex and Opus 4.6 were being described as qualitatively different from earlier tools
- The jump was not just smarter models, but stronger agentic harnesses: tools, planning, context management with compaction, delegation to subagents
- Coding agents were already being described as able to edit files, test, correct mistakes, and work iteratively like a human programmer


Human organization
CEO
VP Eng
VP Sales
Manager
Manager
Manager
IC
IC
IC
IC
IC
IC
AI agent hierarchy
Orchestrator
Sub-agent
Sub-agent
Task
Task
Task
Tool
Tool
Tool
Tool
Tool
Tool
Same organizational logic, different substrate

WHAT CHANGED IN PRACTICE

- The biggest change is not that AI writes code for me
- It is that it removes a huge amount of friction between idea and delivery – scaffolding, building, packaging, etc. are all simple side-effects.
- The focus shifts away from typing and toward planning, steering, and verification
- Work that used to feel too annoying, too expensive, or not worth starting is now worth doing
- So, the result is not less work, but more throughput, more scope, and more things getting done

WHAT CHANGED IN PRACTICE

- The main gain is not just productivity
- The bigger shift is reduced cognitive labor
- AI absorbs the implementation burden and frees attention for higher-level thinking
- What matters more now is synthesis, judgment, taste, and direction
- The work shifts toward choosing what to build and evaluating what is good

WHAT CHANGED IN PRACTICE

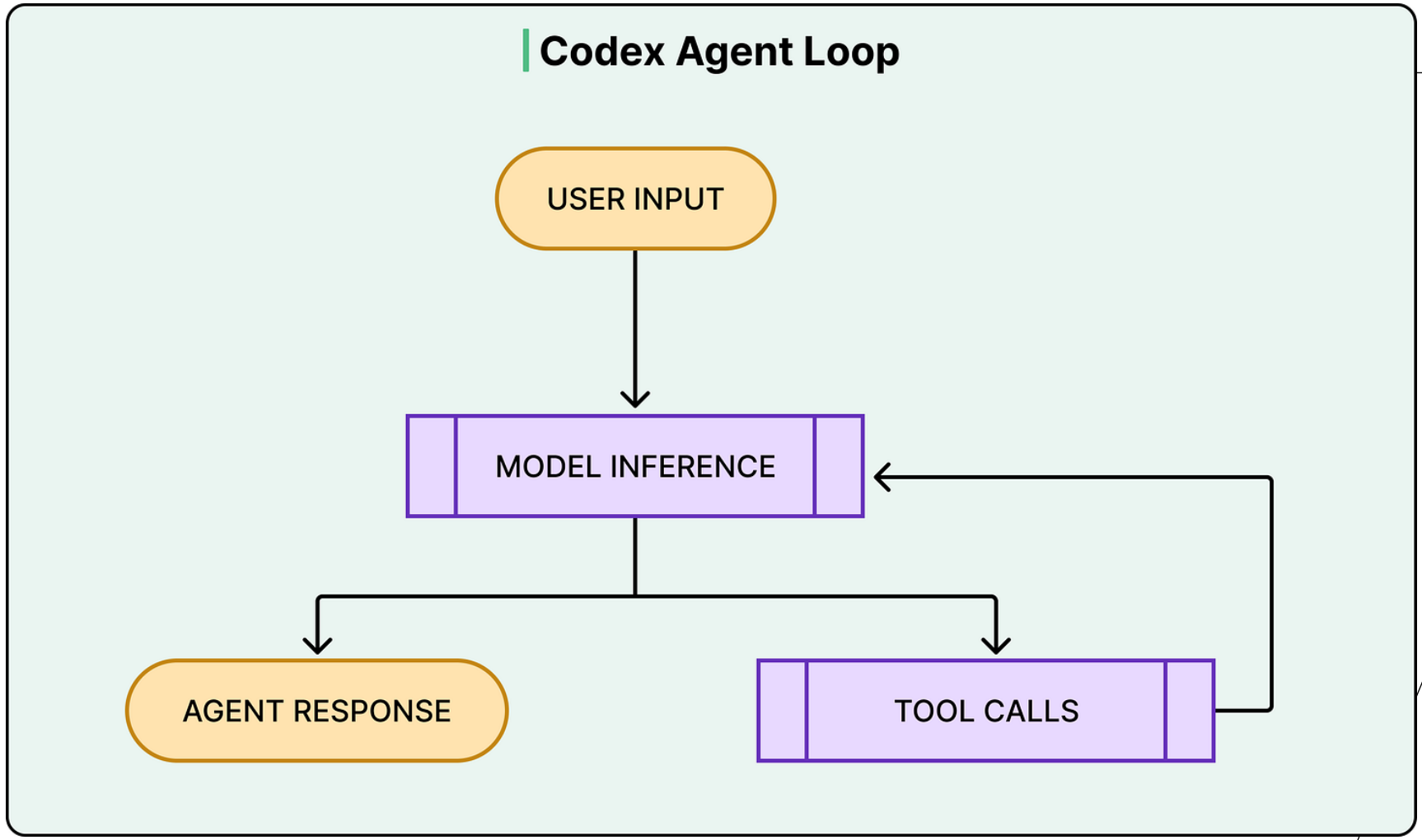
- Projects I had postponed for years suddenly became possible over a weekend
- Library upgrades that used to take days now take hours
- Refactoring or rewriting an entire codebase no longer feels like a major event
- Tech debt is no longer this permanent weight sitting in the backlog

HOW AGENTS WORK

- An agent is a software system, not just a model
- It uses a model plus tools to work through a task iteratively
- It can read code, edit files, run commands, and inspect results
- A CLI, web app, or IDE integration is just a surface for interacting with the agent
- Each task is run in its own isolated cloud/local sandbox
- The repository and context is preloaded into that sandbox
- Multiple tasks can run in parallel
- Progress can be monitored in real time

HOW AGENTS WORK

- A user request becomes a prompt
- The agent uses tools: read files, edit code, run commands, run tests (eg: grep, ls, etc.)
- The model reasons and returns either an answer or a tool call
- The agent executes the tool call and feeds the result back into the model
- This cycle repeats until the task is complete
- A single request can trigger file reads, edits, tests, linting, and multiple passes


Codex Agent Loop
USER INPUT
MODEL INFERENCE
AGENT RESPONSE
TOOL CALLS

HOW AGENTS WORK

- The harness is the system around the model
- It provides tools, context, memory, permissions, and sandboxing
- It manages execution, observations, and when the loop should stop
- This is why agent performance depends on workflow and system design, not just raw model quality

TOOLING LANDSCAPE

- At work, I use GitHub Copilot with Claude and Codex mainly
- It works in VS Code, on the web, in JetBrains/PyCharm, and through the Copilot CLI
- The main limitation is credits
- Personally, I use Claude, Codex and Gemini via Codex, Claude Code and Google Antigravity – through native desktop apps or CLI tools

TOOLING LANDSCAPE

- The model is only part of the experience; the interface matters just as much
- Web interfaces are generally slower and more error-prone
- Local development interfaces work better for real coding
- From my experience, VS Code is the best overall interface
- Codex desktop is second best
- I used to work heavily in PyCharm and JetBrains, but I have been moving toward more AI-friendly editors because editor choice now affects productivity much more than it used to

TOOLING LANDSCAPE

- If privacy matters, I can run open-source models locally through LM Studio or local LLaMA
- That gives access to models like GPT-OSS, DeepSeek, and Qwen
- These local models are not frontier-level, but they are good enough for many coding tasks when data locality matters

MODEL SPECIALIZATION

- You should not use these tools randomly; you have to specialize
- GPT: idea exploration, spikes, research, prompt writing.
- Claude Opus: planning, architecture, orchestration
- Codex: implementation, long refactors, large code changes
- Claude Sonnet: sensible default for everyday coding and bug-fix work
- Haiku / Flash / mini-tier models: cheap, high-frequency tasks
- The smartest model is not always the best model for the job

MODEL SPECIALIZATION

- A recurring pattern in the field is planner/executor/reviewer separation
- Opus handles planning, context gathering, tool-heavy exploration, and codebase explanation
- Codex handles large-scale implementation and often produces fewer subtle bugs
- Several workflow reports recommend starting with Claude for planning, then switching to Codex for execution

MODEL SPECIALIZATION

- Small models are getting good enough for more specific tasks
- Haiku, Mini and Flash-class models are useful for cheap, high-frequency queries
- In some workflows, faster local models can outperform stronger frontier models on time-to-result
- Cost control is part of model choice, not just capability
- Context quality and workflow design still matter as much as the model itself

WORKFLOW OVERVIEW

- Think -> Plan -> Critique -> Execute -> Review
- I start with OpenAI’s GPT, not to write code, but to think out loud
- I refine the goal, clarify constraints, and define what “done” means
- Then I have it generate a prompt for Claude Opus
- Opus turns that into a real local spec and implementation plan, the two most important documents.
- The plan becomes the contract for execution, the spec is used for reference

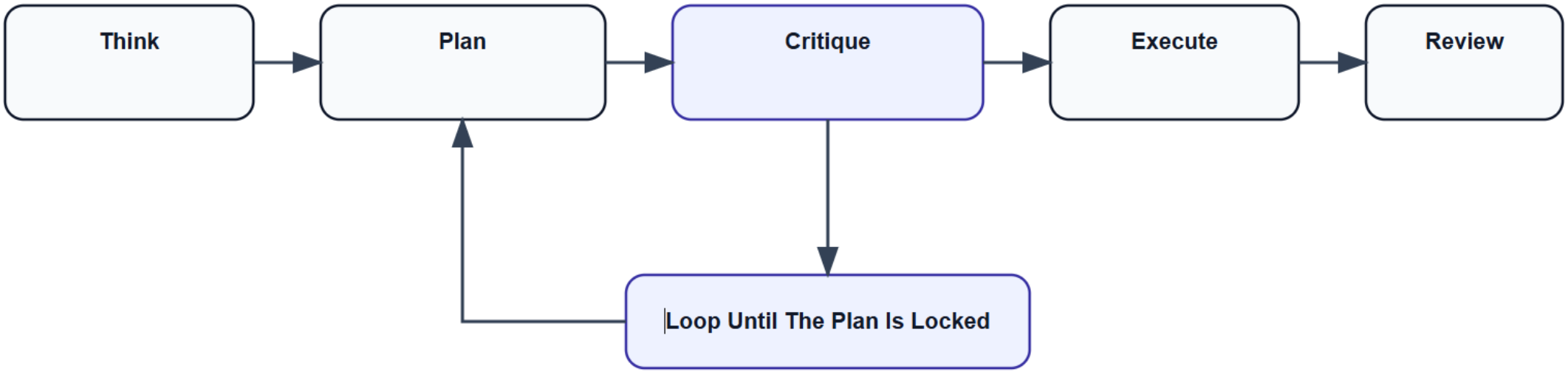
WORKFLOW OVERVIEW

- I do not jump from plan to code
- I critique it
- Then, I send the plan to GPT and sometimes Gemini for critique, just to have a second pair of eyes.
- The loop is: plan -> my critique -> GPT/Gemini critique -> refine -> repeat
- I keep going until the plan is fully specified down to files, methods, variable names, and order of changes
- For production work, this is the high-discipline part of the workflow

WORKFLOW OVERVIEW

- Once the plan is locked, I switch to Codex
- I tell it to follow the plan exactly
- No divergence. No creativity. No architecture changes
- Just execute what is written
- For long-running work, I rely on specs, plans, handoff logs, and context documented in markdown so the task can survive context limits and resume cleanly
- Documentation is a really important part of the workflow

Workflow Overview



VALIDATION + REVIEW

- I do not merge blindly after the agent finishes
- I run the real build locally
- I run the real tests locally
- If something fails, I paste the exact error back
- Never paraphrase logs
- With proper documentation of the product’s dependencies and CI pipeline, this can be automated as well

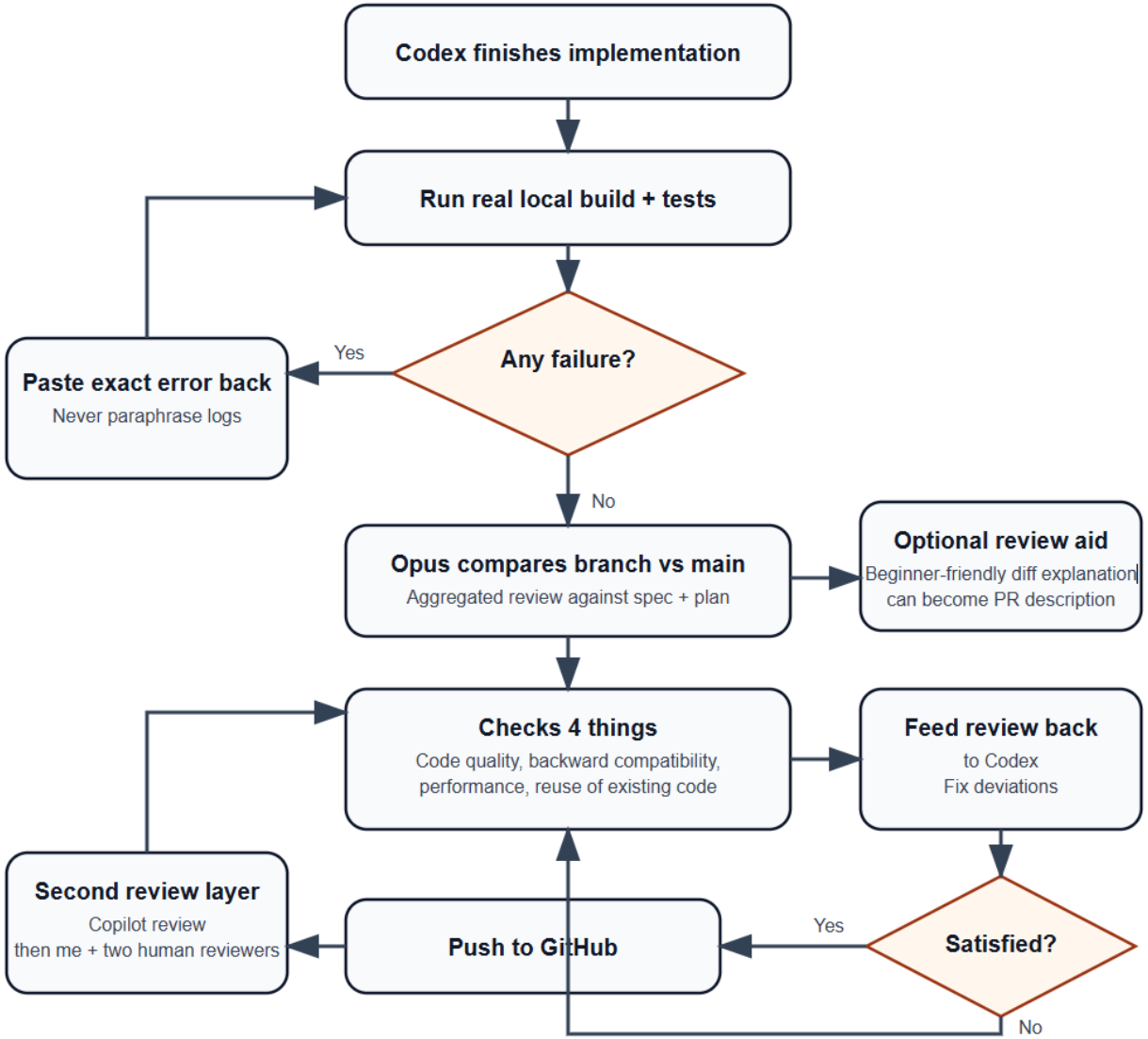
VALIDATION + REVIEW

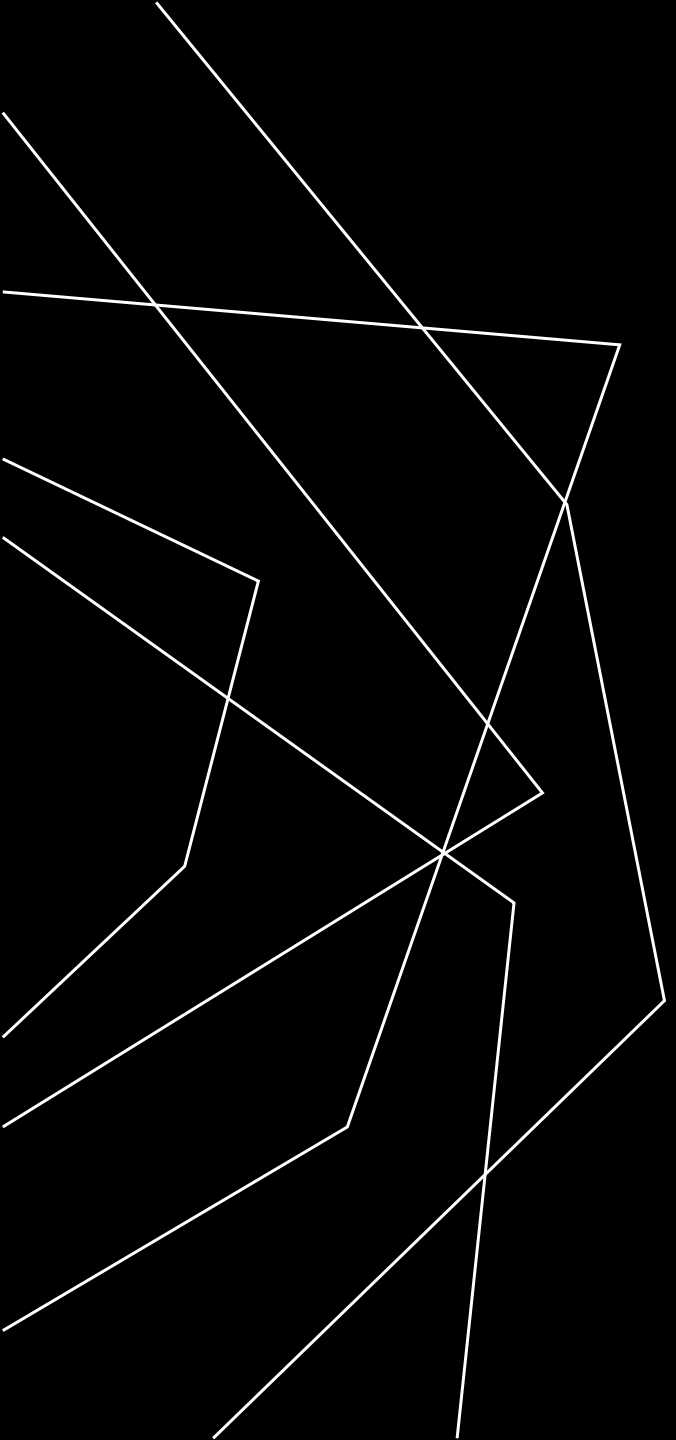
- I use the planning model (Opus) again after implementation, not just before it
- I ask it to compare the current branch against main
- I ask for an aggregated review of all changes in the branch against the previously generated spec and plan
- I also ask for reviewing 4 things: Code quality, backwards compatibility, performance and proper reuse of existing code
- I then feed this review back to the execution session (Codex) and ask it to fix the deviation
- Rinse and repeat until satisfied (probably 2-3 iterations)

VALIDATION + REVIEW

- I push to GitHub and then have Copilot on GitHub do a second review (uses a different model - so a new set of eyes)
- Two human reviewers review it, after I do
- Sometimes I ask Opus, while it is reviewing, to generate a beginner-friendly line-by-line explanation of the diff, then I read that document
- I also make that my PR description so others can review it quickly
- Some phases are probably not as important for quick research projects but helps to be thorough about what you are merging
- As the models get better and start to build ‘trust’, review processes can be made a little lenient

Validation + Review





DEMO

BEST PRACTICES

- Keep sessions focused
- One feature per session
- One bug per session
- No unrelated stacked changes
- Break implementation plans into phases so review remains tractable
- Ask the agent to stop and get confirmation after each phase
- If there are conflicting decisions or ambiguity in a plan, ask the agent to stop and ask for guidance
- Force the agent to stick to the plan during implementation phases

BEST PRACTICES

- Break implementation plans into phases so review remains tractable
- Ask the agent to stop and get confirmation after each phase
- If there are conflicting decisions or ambiguity in a plan, ask the agent to stop and ask for guidance
- Force the agent to stick to the plan during implementation phases

BEST PRACTICES

- Automate the boring parts, but keep decision-making and analysis-heavy parts human
- Don’t use ‘plan mode’. Agent mode is more human-like and succinct in responses than ‘plan’ and ‘chat’ mode
- Be clear in what you are going to do. If your thoughts are confused, the model cannot reliably invent the missing clarity
- Have Q&A sessions, make decisions and dump the conversation in a decisions document

BEST PRACTICES

- Interaction with AI is measured in tokens/words: prompts, context windows, and outputs
- Context windows matter; when the context gets too full and compaction happens, output quality degrades
- You usually watch the context window and expect compaction around roughly 85%
- In theory you can often let it compact twice, but after that the output degrades more noticeably
- Your rough rule is to let compaction happen once at most, then ask for a summary and a handoff, start a fresh session, and have the new session compare the current branch to main to get the summary of changes done so far

BEST PRACTICES

- Do not burn tokens on muscle-memory work
- Do not use agents for git operations, file renaming, package installs, simple find-and-replace, reading logs, checking environment variables, etc

BEST PRACTICES

- Spend more time on the implementation plan if you want to reduce review time later.
- Specification work was never meant to be a time-saving device.
- Specification work is supposed to be harder than coding because it forces a contemplative, critical pass before the bias-to-action phase.
- When everything is marked important, nothing is actually prioritized
- Create reusable workflows rather than relying on ad hoc prompts

BEST PRACTICES

- Express a need for better algorithmic and space complexity while planning.
- Agents automatically adapt and start recommending similar improvements sometimes.
- The models mirror the engineer. If you’re shallow, the output is shallow. If you’re rigorous, the output improves.

BEST PRACTICES

- Context is key; give the model enough relevant context
- AGENTS.md and ARCHITECTURE.md are important for seed context
- If your repository has inconsistencies, document them in AGENTS.md
- SPEC.md and IMPLEMENTATION_PLAN.md – the outputs of the planning phase, augment the existing context
- The best AGENTS.md is a map, not an encyclopedia
- A giant instruction file rots quickly as rules go stale
- Use AGENTS.md as an index with links to other docs

BEST PRACTICES

- Use features like ‘add-to-queue’, ‘steer’, and ‘stop-and-send’
- Watching the live action stream matters sometimes, to interfere if the agent is drifting in the wrong direction
- If the agent is wasting time on something unimportant, intervene and redirect it
  - Example: switching from PDF as the preferred output mode to Markdown/text when PDF generation on Windows becomes a time sink
- Use subagents when the platform supports them
- Use multiple IDE instances for cross-repo collaboration, generate analysis reports of code and share across agents.
  - Eg: Share code analysis reports of the core product to agent sessions building a feature in a client library that depends on the core, to provide enough context.
- Provide visual references to the agent for UI work. Eg: screenshots, images etc.

BEST PRACTICES

- Use sandboxes if you are going all-in in terms of agent permissions. (VMs, Docker, etc).
- Ideally, one should write tests manually or at least the spec for those tests and then ask agents to generate from it. You know the business/domain more than the agent. The agent only knows to test the code it wrote.
- Track all prompts used to build a feature or fix, in the source repo, so that the thought process is preserved.
- For large refactors/rewrites (Eg: adding types to an entire codebase), use a hand-off log and run agents sequentially.
- For concurrent tasks, use a sync-log and run agents asyncly.

BEST PRACTICES

- Use the SingleFile Chrome extension to export conversations from web-based chats as single blobs into local repos for book-keeping and agent context.
- Your voice is roughly 3x faster than your keyboard. Use tools like Handy and OpenWhispr for voice dictation through local models.
- Use git worktrees for parallel agent work.

Plain Text

```
AGENTS.md
ARCHITECTURE.md
docs/
├── design-docs/
│   ├── index.md
│   ├── core-beliefs.md
│   └── ...
├── exec-plans/
│   ├── active/
│   ├── completed/
│   └── tech-debt-tracker.md
├── generated/
│   └── db-schema.md
├── product-specs/
│   ├── index.md
│   ├── new-user-onboarding.md
│   └── ...
├── references/
│   ├── design-system-reference-llms.txt
│   ├── nixpacks-llms.txt
│   ├── uv-llms.txt
│   └── ...
├── DESIGN.md
├── FRONTEND.md
├── PLANS.md
├── PRODUCT_SENSE.md
```

WHAT AI IMPROVES

- The process from intent to delivery is significantly compressed
- Once the plan is good, implementation often only takes minutes
- It frees more attention for product judgment and UX decisions
- Search and exploration is extremely fast and cheap (esp. deep research)
- You learn a lot by reviewing AI-generated code, patterns, and techniques
- Easier codebase understanding and repo exploration
- One-off scripts, shell scripts, and utility automation become cheap to create
- Lowers the barrier to working outside your core expertise
- Being a generalist pays off even more now

WHAT AI MAKES HARDER

- AI can produce too much code too quickly
  - Large diffs are harder and take longer to review than code you wrote yourself
- It is tempting to accept output because the agent sounds confident esp. when you don’t know the stack.
- Cognitive load can get too high
  - It is easy to run multiple projects at once just because the agents make parallel work possible
  - Constant context switching
- The workflow is useful, but it needs self-discipline
  - When you automate all of it including the planning, there is no human thinking involved.

WHAT AI MAKES HARDER

- Long sessions degrade as context windows fill up
- Handoffs, compaction, and session resets become part of the job
- Multi-agent or multi-session workflows make fault tracing harder
- Debugging becomes harder when the implementation path is spread across plans, prompts, agents, and review loops
- You know the code less deeply afterward because you did not type it
- Agents may tend to recreate existing patterns and skip reuse of code causing drift

RISKS AND ACCOUNTABILITY

- Higher output can quickly turn into higher expectations
- As you generate more over time, your responsibility across the codebase also grows
- People start juggling more work because the tools make it feel possible
- More multitasking and parallel work can increase burnout risk
- Quality can slip if teams are not disciplined in code review

RISKS AND ACCOUNTABILITY

- Blurred work / non-work boundaries
  - small prompts slip into lunch, meetings, evenings, and early mornings
  - work becomes ambient and easier to keep advancing
  - downtime stops feeling like real recovery
- It expands the amount of work one person can realistically attempt
- People start doing work they would previously have deferred, outsourced, or avoided
- AI fills knowledge gaps, so job scope widens

AI And Machine Learning

AI Doesn't Reduce Work—It Intensifies It

by Aruna Ranganathan and Xingqi Maggie Ye

February 9, 2026

Illustration by Eynon Jones

RISKS AND ACCOUNTABILITY

- AI can help produce the work but it does not own the result
- Engineers are still accountable when something breaks
- Passing tests do not remove the need for ownership
- The speed changed, but responsibility did not
- Do not treat “tests passed” and “AI reviewed it” as sufficient by default
- Human review is still the decision point for what is safe to merge
- The same system should not be the only author and judge of its own work

AI NEWS TECH

Amazon blames human employees for an AI coding agent's mistake

Image: The Verge

/ Two minor AWS outages have reportedly occurred as a result of actions by Amazon's AI tools.

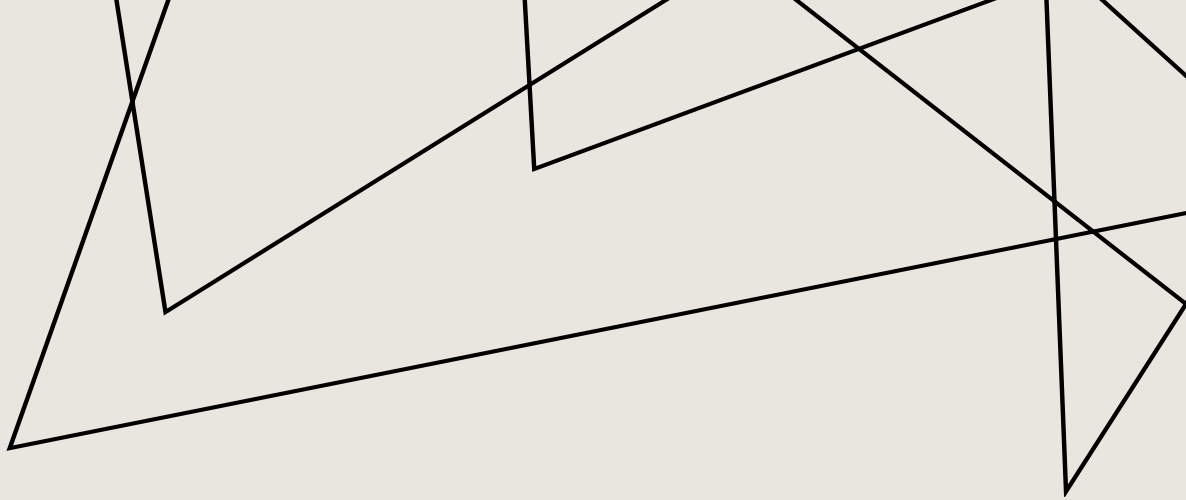
by Robert Hart
Feb 20, 2026, 11:52 AM EST

11 Comments (All New)

WHAT THIS MEANS FOR US

- The value shifts upward from typing to planning, judgment, and review
- The hard part shifts toward architecture, constraints and design.
- Automatic code generation strengthens the need for product judgment and decision-making.
- Breadth, adaptability, and cross-functional range become more valuable
- More work becomes feasible, so expectations and standards will both rise

WHAT THIS MEANS FOR OUR TEAM

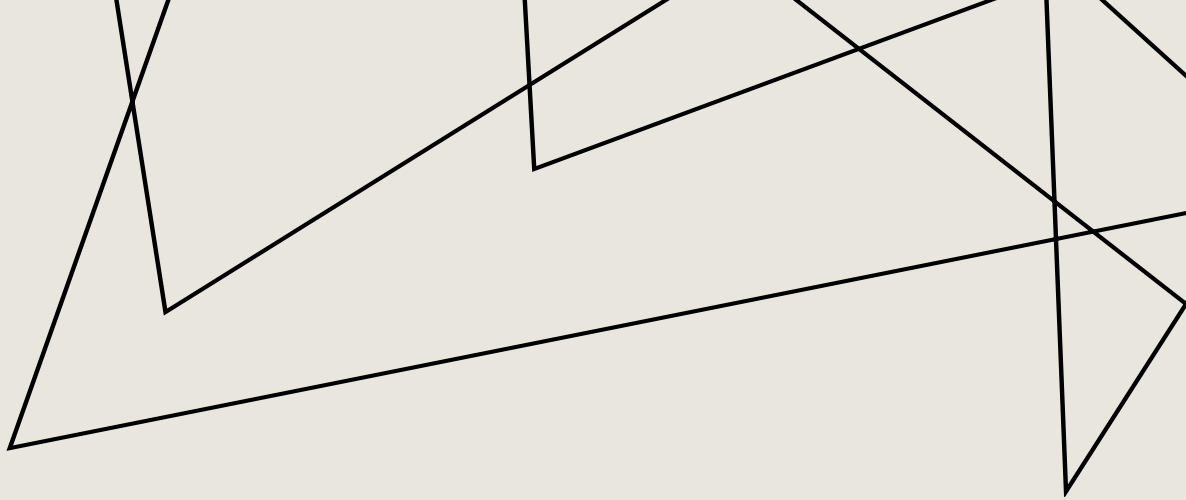


- Smaller, flatter teams may become more common
- Teams will need norms for code review, ownership, and sustainable pace
- Engineers, PMs, and designers start working in tighter loops, with more overlap in who explores, prototypes, and ships
- The better the engineer, the better the output from the model
- The gap between disciplined teams and average teams may widen

WHAT THIS MEANS FOR OUR COMPANY

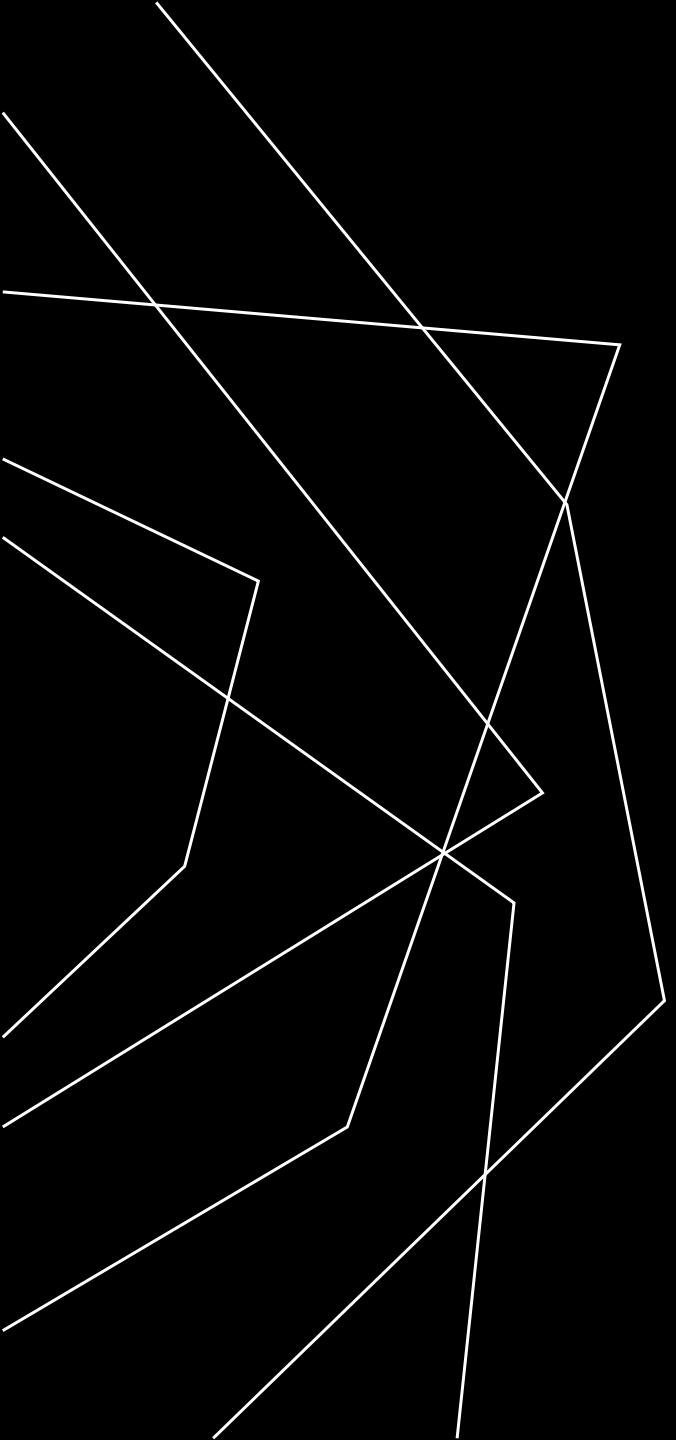
- AI support is becoming part of product value, not just engineering productivity
- If our tools and workflows do not adapt, people will look for alternatives that do
- Competitive advantage shifts more towards real-world assets that are harder to copy:
  - Customer trust and relationships
  - Brand
  - Infrastructure
  - Proprietary data
  - Deep workflow integration
  - Durability of systems

CLOSING



- Intelligence is becoming a commodity on tap
- The scarce thing is no longer raw cognitive horsepower
- What remains scarce is taste, direction, synthesis, and judgment


Some day we won't even need coders any more. We'll be able to just write the specification and the program will write itself
Oh wow, you're right! We'll be able to write a comprehensive and precise spec and bam, we won't need programmers any more!
Exactly!
And do you know the industry term for a project specification that is comprehensive and precise enough to generate a program?
Uh... no....
Code
It's called code
CommitStrip.com



THANK YOU